Advanced Combinatorics

Burnside/Pólya, Möbius inversion, generating functions, sublinear multiplicative sums

Learning goals

- Count objects **up to symmetry** with Burnside's lemma and Pólya's cycle index
- Combine Burnside with constraints (adjacency, colour counts) via fixed-point counting
- Use **Möbius inversion** and Dirichlet convolution fluently; evaluate sums with floor blocks
- Model counting problems with **generating functions** (OGF/EGF, exp/log)
- Compute $\Sigma\, f(i)$ for $i \le 10^{10}$–10^{11} in sublinear time: **Du's sieve**, **Lucy/min-25**

Where we are

- Week 30: inclusion–exclusion, nCr, expectation
- Regional-07: CRT, φ, Möbius intro
- WF-02: NTT, polynomial inverse / log / exp — the *engine* for generating functions
- **Today:** the theory that tells you *what* to compute
- WF-level counting problems combine 2–3 of these tools in one solution

Motivating problem

> Count necklaces of n = 10^9 beads with k colours, where rotations are considered the same. Answer mod $10^9 + 7$.

- k^n colourings, but the classes have **different sizes** (`aaaa` has 1 rotation, `abab` has 2, `aabb` has 4)
- Dividing by n is wrong
- Need: count **orbits** of a group acting on a set

Group actions in one slide

- A group G acts on a set X: each $g \in G$ permutes X
- **Orbit** of x: $\{ g \cdot x \}$ — the "objects up to symmetry"
- **Stabiliser** of x: $\{ g : g \cdot x = x \}$
- **Fix(g)** = $\{ x : g \cdot x = x \}$
- Orbit–stabiliser: $|\text{orbit}(x)| \cdot |\text{stab}(x)| = |G|$

Burnside's lemma

$$\#\text{orbits} = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|$$

Proof (double counting pairs (g, x) with g·x = x):

- Σ_g |Fix(g)| = Σ_x |stab(x)| = Σ_x |G| / |orbit(x)|
- Each orbit O contributes |O| · |G|/|O| = |G|
- So the sum is |G| · #orbits ∎

Recipe: **enumerate group elements by type**, count what each one fixes.

Necklaces

- G = rotations by i = 0..n−1; rotation by i has **gcd(i, n) cycles**
- Fixed colourings: constant on each cycle ⇒ k^{gcd(i,n)}
- #i with gcd(i, n) = n/d equals φ(d)

$$N(n,k) = \frac{1}{n}\sum_{d \mid n} \varphi(d)\, k^{n/d}$$

O(√n) divisors, each φ in O(√d) — or factor n once and enumerate divisors with φ.

Bracelets (rotations + reflections)

Dihedral group, $|G| = 2n$. Reflections:

n	reflections	cycles each	fixed
odd	n, axis through 1 bead	(n+1)/2	k^{(n+1)/2}
even	n/2 through 2 beads	n/2 + 1	k^{n/2+1}
even	n/2 through 0 beads	n/2	k^{n/2}

$B(n, k) = (n \cdot N(n,k) + \Sigma_\text{reflections fixed}) / 2n$

POJ 2409 *Let it Bead* is exactly this.

Burnside with constraints

"Adjacent beads must differ" — still Burnside, but Fix(g) changes:

- Rotation with g = gcd(i, n) cycles fixes colourings with period g
- Valid ⇔ the first g beads form a **proper colouring of a cycle C_g**
- #proper colourings of C_g = (k−1)^g + (−1)^g (k−1)

$$\frac{1}{n}\sum_{d\mid n}\varphi(n/d)\left[(k-1)^d+(-1)^d(k-1)\right]$$

General pattern: Fix(g) = #valid objects on the **quotient** structure (often a transfer-matrix trace: tr(T^d)).

Pólya: counting with weights

Cycle index $Z(G) = (1/|G|)\ \Sigma_g\ \Pi_j\ a_j^{c_j(g)}$, $c_j(g)$ = #cycles of length j.

- Substitute $a_j = x_1^j + x_2^j + \ldots + x_k^j$ ⇒ coefficient of $x_1^{e_1}\ldots x_k^{e_k}$ = #orbits with exactly e_i beads of colour i
- Example: necklaces with exactly r red beads out of n: $(1/n)\ \Sigma_{d \mid \gcd(n,r)}\ \varphi(d)\ C(n/d, r/d)$
- Under one cycle of length j all beads share a colour ⇒ colour counts come in multiples of j

When the group is irregular

Cube faces, graph automorphisms, grid symmetries:

1. Write each generator as a permutation of the positions
2. Close under composition with BFS ⇒ the whole group (cube rotations: 24)
3. For each element, compute its cycle type
4. Apply Burnside/Pólya

Group size ≤ a few thousand: brute closure is fine. UVa 10601 *Cubes*: 12 edges, 24 rotations, colour multiplicities.

Dirichlet convolution and Möbius

$(f * g)(n) = \sum_{d \mid n} f(d)\, g(n/d)$

Identity	Meaning
$\mu * 1 = \varepsilon$	$\sum_{d \mid n} \mu(d) = [n = 1]$
$\varphi * 1 = \mathrm{id}$	$\sum_{d \mid n} \varphi(d) = n$
$\mathrm{id} * \mu = \varphi$	
$1 * 1 = d$ (divisor count)	

Möbius inversion: $g = f * 1 \Leftrightarrow f = g * \mu$.

Practical form: replace $[\gcd = 1]$ by $\sum_{d \mid \gcd} \mu(d)$.

The gcd-sum pattern

$$\sum_{i \le n} \sum_{j \le m} [\gcd(i, j) = 1] = \sum_{d \ge 1} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

- ⌊n/d⌋ takes O(√n) distinct values ⇒ group d into **blocks** [l, r] with r = n / (n / l)
- Needs prefix sums of μ — linear sieve to 10^7, or Mertens function (Du's sieve) beyond
- Σ gcd(i, j) = Σ_d φ(d) ⌊n/d⌋⌊m/d⌋ (since id = φ ∗ 1)

```
for (ll l = 1, r; l <= min(n, m); l = r + 1) {
    r = min(n / (n / l), m / (m / l));
    ans += (M[r] - M[l - 1]) * (n / l) * (m / l);
}
```

Generating functions — the toolkit

Structure	OGF / EGF operation
disjoint union	A + B
sequence / pair	A · B (unlabelled: OGF; labelled: EGF)
sequence of ≥0	1 / (1 − A)
multiset (unlabelled)	Euler transform exp(Σ A(x^k)/k)
set (labelled)	exp(A)
connected from all (labelled)	log

Coefficient extraction = polynomial inverse / log / exp from WF-02, O(n log n).

Generating functions — classics

- **Partitions:** $\prod 1/(1-x^k) = 1 / \sum (-1)^k x^{k(3k-1)/2}$ (pentagonal) ⇒ $p(n)$ in $O(n\sqrt{n})$ or one inverse
- **Connected labelled graphs:** $C(x) = \log(\sum 2^{C(n,2)} x^n/n!)$
- **Binary trees with weights in a set S** (CF 438E): $F = 1 + S \cdot F^2$ ⇒ $F = 2 / (1 + \sqrt{1 - 4S})$
- **Rational GF ⇔ linear recurrence** — Berlekamp–Massey (WF-02) finds it from terms

Think "what is the GF?" before "what is the DP?"

Sublinear sums: the setting

Want Σ_{i ≤ n} f(i) for multiplicative f, n up to 10^{10}–10^{11}.

Method	Needs	Time
Linear sieve	—	O(n) — too slow
Du's sieve	g with f ∗ g, g easy to sum	O(n^{2/3})
Lucy (min-25 part 1)	f(p) a polynomial in p	O(n^{3/4} / log n)
min-25 part 2	f(p^e) computable	≈ O(n^{3/4} / log n) in practice

All of them work on the set V = { ⌊n/i⌋ } of size ≈ 2√n.

Du's sieve (Dirichlet hyperbola)

If h = f ∗ g with easy prefix sums H, G (and g(1) = 1):

$$F(n) = H(n) - \sum_{d=2}^{n} g(d)\, F(\lfloor n/d \rfloor)$$

- φ ∗ 1 = id ⇒ Φ(n) = n(n+1)/2 − Σ_{d≥2} Φ(⌊n/d⌋)
- μ ∗ 1 = ε ⇒ M(n) = 1 − Σ_{d≥2} M(⌊n/d⌋)
- Sieve directly up to n^{2/3}, memoise the rest ⇒ **O(n^{2/3})**

Du's sieve code

```
ll Phi(ll x) {                          // sum_{i<=x} phi(i) mod MOD
    if (x <= lim) return sphi[x];       // linear sieve up to n^{2/3}
    auto it = memo.find(x);
    if (it != memo.end()) return it->second;
    ll r = (ll)((__int128)x * (x + 1) / 2 % MOD);
    for (ll l = 2, rr; l <= x; l = rr + 1) {
        rr = x / (x / l);
        r = (r - (rr - l + 1) % MOD * Phi(x / l)) % MOD;
    }
    return memo[x] = (r + MOD) % MOD;
}
```

Store memo in an array indexed by n / x instead of a hash map for a 2–3× speedup.

Lucy's prime counting

g(v) = #{ 2 ≤ i ≤ v : i prime or smallest prime factor > p } for v ∈ V.

- Start: g(v) = v − 1 (everything is "prime")
- For each prime p ≤ √n, remove numbers whose smallest prime factor is p:

$$g(v) \mathrel{-}= g(\lfloor v/p \rfloor) - g(p-1) \qquad (v \geq p^2)$$

- At the end g(v) = π(v) for **every** v ∈ V
- Same recurrence with i^k weights ⇒ Σ_{p ≤ v} p^k

Lucy code

```
struct Lucy {
    ll n, sq; vector<ll> vals, cnt;            // vals = n/1, n/2, ... (descending)
    int idx(ll v) const { return v <= sq ? (int)vals.size() - (int)v : (int)(n / v) - 1; }
    Lucy(ll n_) : n(n_) {
        sq = sqrtl(n); while (sq * sq > n) sq--; while ((sq + 1) * (sq + 1) <= n) sq++;
        for (ll i = 1; i <= n; i = n / (n / i) + 1) vals.push_back(n / i);
        for (ll v : vals) cnt.push_back(v - 1);
        for (ll p = 2; p <= sq; p++) {
            if (cnt[idx(p)] == cnt[idx(p - 1)]) continue;      // p composite
            ll before = cnt[idx(p - 1)];
            for (size_t i = 0; i < vals.size() && vals[i] >= p * p; i++)
                cnt[i] -= cnt[idx(vals[i] / p)] - before;
        }
    }
    ll pi(ll v) const { return cnt[idx(v)]; }  // v must be in V
};
```

$\pi(10^{10}) = 455052511$ in well under a second.

min-25 part 2: the multiplicative sum

S(x, j) = Σ f(i) over 2 ≤ i ≤ x with smallest prime factor ≥ p_j.

$$S(x,j) = \underbrace{\big(G(x) - G(p_j - 1)\big)}_{\text{primes}} + \sum_{k \ge j,\ p_k^2 \le x} \ \sum_{e \ge 1,\ p_k^{e+1} \le x} \Big(f(p_k^e)\, S(\lfloor x/p_k^e \rfloor, k+1) + f(p_k^{e+1}) \Big)$$

- G = prime sums of f from part 1 (f(p) a polynomial ⇒ one Lucy table per monomial)
- Answer = S(n, 0) + f(1)
- Plain recursion, no memo — the recursion tree is small (≈ n^{3/4}/log n nodes)

Choosing the tool

Task	Tool
π(n), Σ primes ≤ n	Lucy
Σ φ, Σ μ	Du (simpler) or min-25
Σ f with f(p^e) weird (e.g. σ_k, "p xor e")	min-25
count n ≤ N with exactly 4 divisors (CF 665F)	Lucy + small case analysis
Σ_{i,j} gcd-type double sums	Möbius + floor blocks + Du for prefix sums

Always check n: 10^7 ⇒ just sieve. 10^{11} ⇒ sublinear.

Pitfalls

- Burnside: averaging **mod p** needs the inverse of $|G|$ — fine for prime MOD, wrong if $|G| \equiv 0 \pmod p$
- Bracelets: even/odd n reflection cases swapped
- Floor blocks: `r = n / (n / l)` must also be capped by m in two-variable sums
- `x * (x + 1) / 2` overflows at $x = 10^{10}$ — use `__int128` or reduce mod before multiplying (and divide first)
- Lucy with float `sqrt`: fix `sq` with integer loops
- min-25: forgetting the `+ f(p^{e+1})` term (prime powers have no larger factor to recurse into)

Practice problems

- CSES **2209** *Counting Necklaces*, CSES **2210** *Counting Grids* — Burnside
- POJ **2409** *Let it Bead*; POJ **2154** *Color* (n up to 10^9) — Burnside + φ
- UVa **10601** *Cubes* — Pólya on a small irregular group
- SPOJ **VLATTICE**, CF **803F** *Coprime Subsequences*, CF **547C** *Mike and Foam* — Möbius
- CF **1559E** *Mocha and Stars* — Möbius + knapsack
- CF **438E** *The Child and Binary Tree* — GF + polynomial sqrt/inverse
- Library Checker *Counting Primes*, *Sum of Totient Function*; CF **665F** *Four Divisors*; LOJ **6053** — sublinear sums

Summary

- Burnside: average the number of fixed objects over the group; constraints change only Fix(g)
- Möbius: turn [gcd = 1] into Σ μ(d); evaluate with floor blocks
- Generating functions decide *what* to compute; WF-02 polynomials compute it
- Du's sieve O(n^{2/3}); Lucy/min-25 ≈ O(n^{3/4}/log n) on the ⌊n/i⌋ grid

Next: Lecture 8 — eertree, suffix trees, Lyndon words, periodicity